

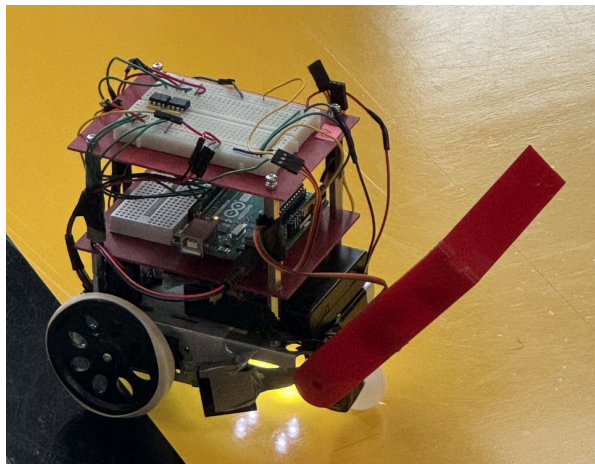
Group 15

Joshua Baker jwb388

Alma Meili Ren arr254

Richard Orellana rao92

1. Robot Design and Strategy Overview



The mechanical design of the robot is very simple, not deviating much from the basic top-view outline of the chassis. There are two independent DC motors which rotate rubber-lined plastic wheels. The wheels are not connected in any way by an axle or another sort of connecting bar. The small ball at the front acts like a third wheel, which allows the robot to move without pushing the chassis along the ground. The different components are located on vertical layers, a design decision that was initially made to accommodate the placement of the color sensor on the underside of the

chassis. This tiered design raised the center of gravity of the robot, which came with unforeseen consequences during the competition. An acrylic arm attached to a miniservo to the front of the robot acts as the main collection apparatus, replacing the original cardboard arm that was used in Milestone 4. Duct tape and electrical tape were heavily used to keep all the wires in place.

While the color sensor and line sensors were implemented into the design for the milestones, they were not included in the final design to reduce unnecessary wires and tangling. All other electrical components were connected to an Arduino UNO board which was powered by a 9V battery. Two L9110 H-bridges were used to send signals to the DC motors. A 6V battery pack containing four 1.5V AA batteries was used as the power source for a PWM servo motor.

The final movement pattern of the robot was hard-coded. From activation, it moves forward until reaching the color border then lowers the arm. The time it took to reach the border was determined through trial-and-error and the arm-lowering action is performed based on that time. Then, the robot does a jagged leftward sweep by turning only one wheel at a time. Once the cube-collecting sweep is done, the robot moves forward a predetermined amount and completes a large arc which makes it face the border once again. From this position, it repeats the process above two more times. A simplified diagram describing this process is shown in Appendix D.

2. Design Process Reflection.

Milestone 2 was fairly straightforward. Our wiring was simple and assembling the robot proved to be quick. To save time, we calibrated each type of motion the robot would use. This way, we simply input the necessary distances and angles into our code to follow the prescribed path.

Milestone 3 was not straightforward. Because we were not given a second QTI sensor and never informed that we were supposed to have a second one, this milestone was painfully difficult. With only one QTI sensor, it was hard for the robot to know which direction to follow the line (i.e. was the line to the right or left of the robot). We attempted to solve this problem by involving the color sensor into our code so that when the QTI sensor would activate the color sensor's input would determine which direction the robot should turn. However, the differing positions of the QTI and color sensor impacted their ability to cross-communicate. We spent a great deal of time and stress trying to figure out the optimal positioning for the QTI and color sensors before finding out that we were missing a second QTI sensor. Once we received our second QTI sensor, this milestone was actually quite easy because depending on which QTI sensor was activated the robot knew which side was black. Our edge detection immediately worked. Distinguishing the final black border took some effort. The actual algorithm was not particularly difficult to finish, however the specific delay necessary to have both QTI sensors see black without significantly affecting the normal border detection was finicky.

Milestone 4 in and of itself was not particularly difficult. We hardcoded our motion, so the only effort was in picking the correct distances and angles to achieve the path we wanted. What was difficult was making the servos work. The behavior of the servo was unbelievably inconsistent—the same code with the same wiring on the same servo would produce completely different results. The code we eventually got working on one servo would be completely nonfunctional on another servo, and produce a whining noise on another. We still have no comprehension of why this was happening. Our main method of prototyping included rough cardboard prototyping of the arm and other ideas we were exploring before we settled on the final design.

3. Competition Analysis.

Our robot performed quite badly in the competition. We lost three times and tied twice. Our underlying design worked reasonably well, however the robot kept breaking. This was the primary reason for our disappointing performance. We had to forfeit one round because our arm snapped off and our battery pack was broken. Another round we lost because the servo stopped functioning, for no apparent reason.

A weakness of our robot was its tendency to tip over. Because our design resembled that of a double decker bus, the center of gravity was quite high. A small nudge could push the robot over. On one round the robot tipped over and began violently seizing on the ground. This was funny but did not score us points.

One unexpected benefit of our simple design was that it was able to interfere with the motion of other robots. Since our maximum score across all five rounds was one cube, our ability to prevent the other robots from gathering cubes played an integral role in the few rounds we did not lose. The arm would get stuck inside the perimeter of the opponent robot and prevent their motion almost entirely.

4. Conclusions.

Despite our loss in the competition, the design of our robot should not be considered a failure. During testing, our predetermined path was able to collect more than half of the cubes on the board. In a competition environment, we were unable to replicate this success due to the

presence of another robot. Without any sensors sending data back to the Arduino, our robot would get entangled in the structure of the opposing team's robot. Although this was not the intended behavior of our design, it resulted in a tie every time it occurred. The addition of a secondary collection mechanism could have gotten us the necessary cubes to prevent a tie. The hooked arm initially served as our primary collector but it instead took the role of a preventative structure when it came to the other team keeping their collected cubes. Our robot was able to push robots that focused on speed, resulting in cubes escaping the opponent's perimeter.

Were we to do this project over again, more focus would be put on the cube-collecting structures. Most of our mechanical design time was spent rebuilding the robot to be slightly taller and not much consideration went to what would be attached beyond the chassis. The hooked arm was always going to be part of the design but it relied on a weak servo motor which could not produce much torque and ultimately failed in the competition. A stronger servo for the arm as well as better wheels would have been greatly advantageous. Along with that, a collection mechanism that attached to the side of the robot could have gotten us more cubes. We made unnecessary purchases based on poorly formulated design concepts which greatly reduced our capacity to design a better robot. Our final design stayed within the specified dimensions and left plenty of room for a larger robot.

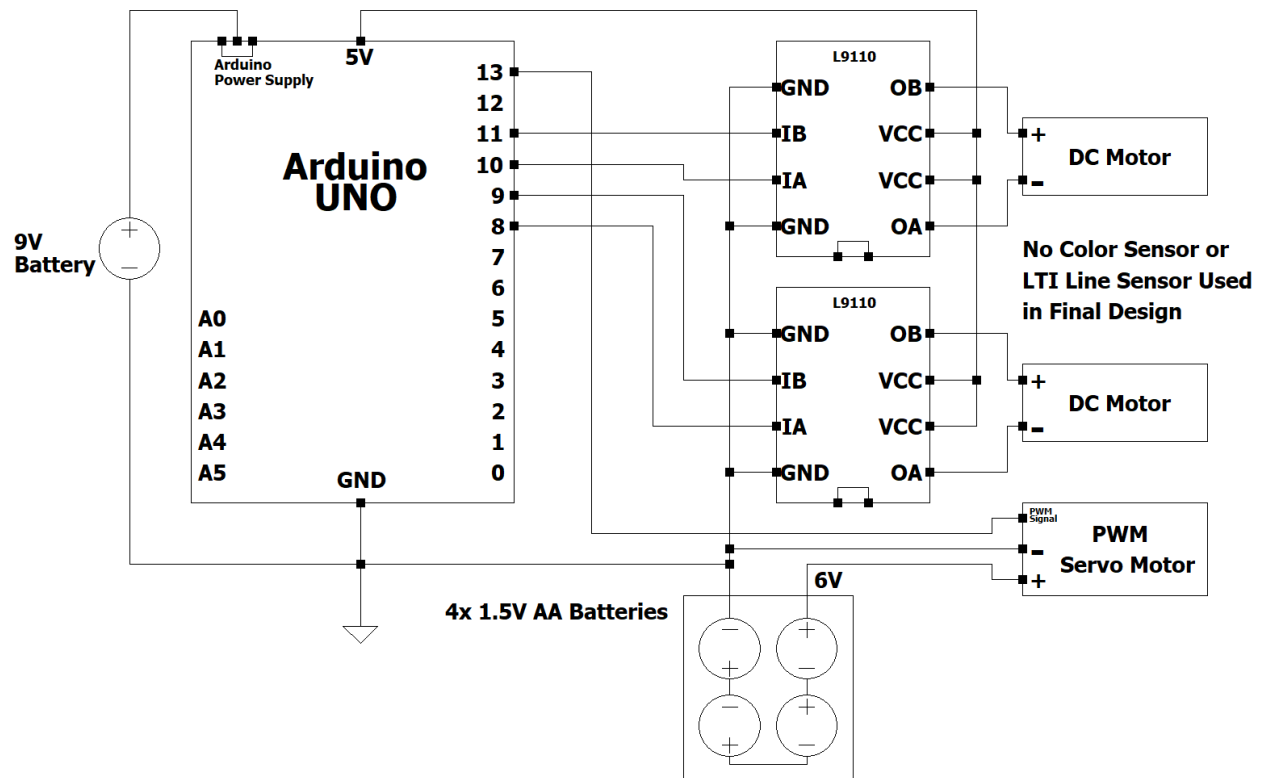
For students doing this next year, prototyping with cardboard and tape will be incredibly valuable. A functional model that can demonstrate the efficacy of a design will save them time and potential material costs. Another important thing for them to know is that not all of the components in the provided kits work as intended. If something is not working as the students know it should, the first thing to try is to replace the component.

While the final goal of this project was to have a competition-ready robot, the true goal was to gain a better understanding of the basics of mechatronics design. Although our robot was not a top contender, the process it took to create it deepened our understanding of both mechanical and electrical design. The four milestones guided our development progress in the right direction while still allowing for diversity in the designs. Our final design accurately represented what the group had envisioned from the start but did not perform in the same manner.

Appendix A: Bill of Materials

Team #15 Bill of Materials					
Part #	Description	Source	Quantity	Unit Cost	Cost
9219T148	Steel Filter Mesh Sheets, 4 x 4 Mesh Size, 0.170" Opening Size 1 ft. x 2 ft.	Mc-Master Carr	1	20.63	20.63
8843K33	Polyester Fabric Black, 61" Wide x 0.022" Thick 1 ft.	Mc-Master Carr	1	8.50	8.50
N/A	Red Acrylic Sheet 1ft. x 1ft.	RPL	1	5.00	5.00
N/A	Arm Piece #1	RPL (laser cut)	1	1.10	1.10
N/A	Arm Piece #2	RPL (laser cut)	1	1.00	1.00
N/A	Wheel	RPL (laser cut)	2	1.46	2.92
				Total Cost	39.15

Appendix B: Circuit diagram



Appendix C: CAD Drawings

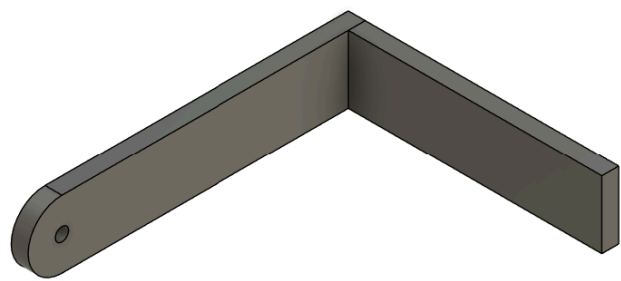


Figure C1. CAD Model of arm.

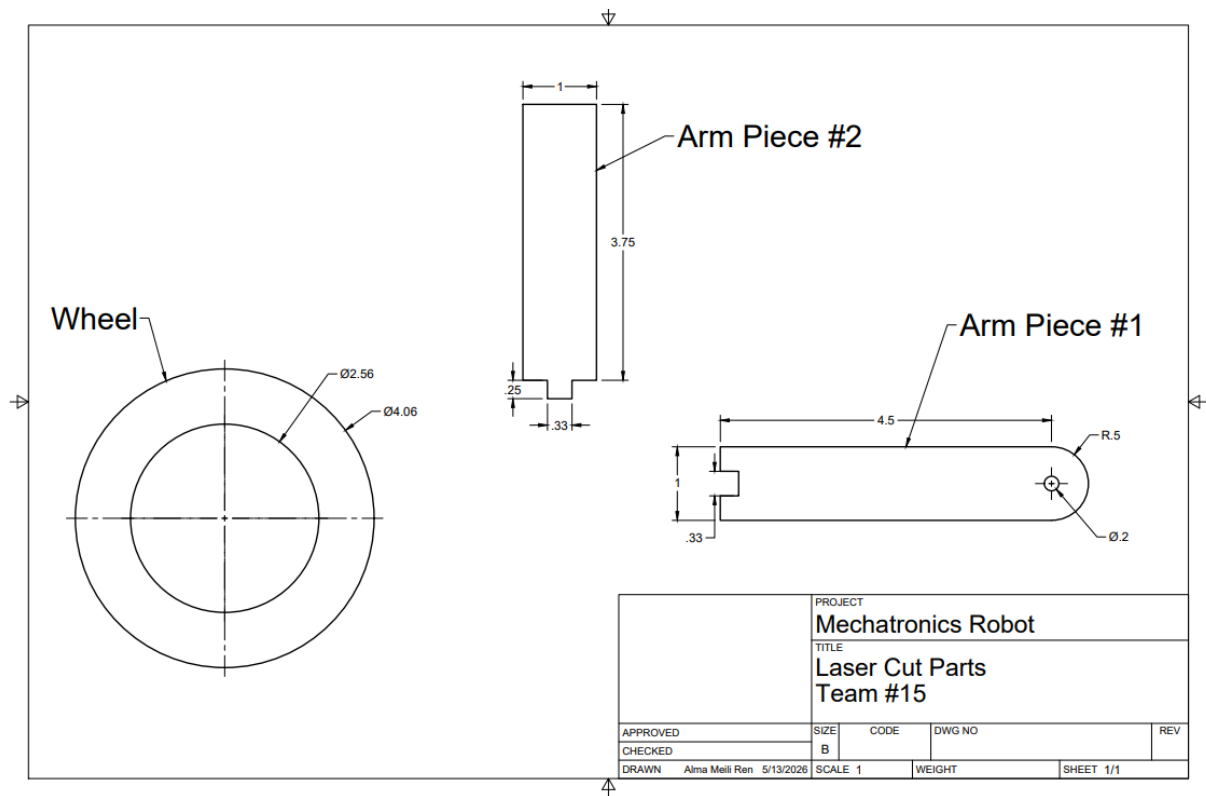


Figure C2. Drawing of parts with dimensions.

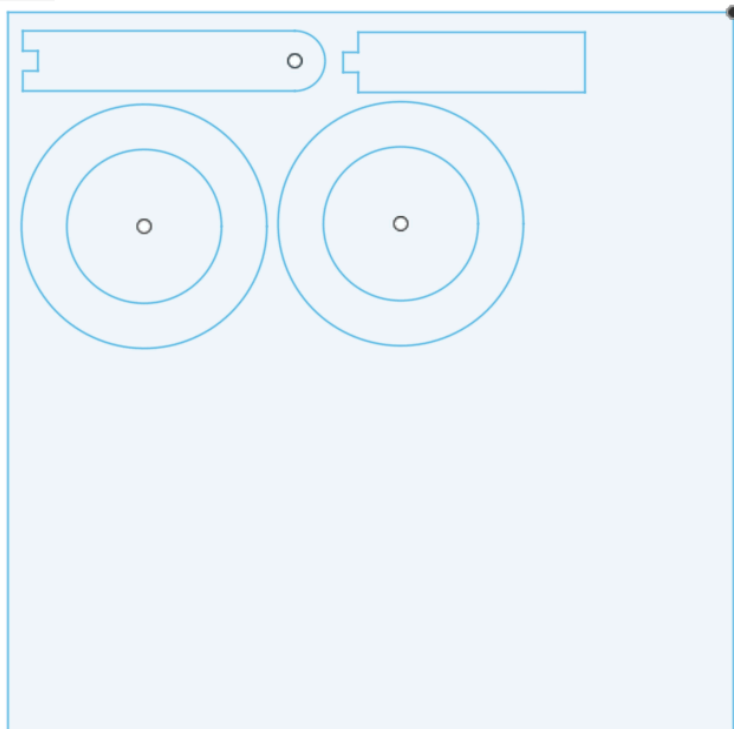
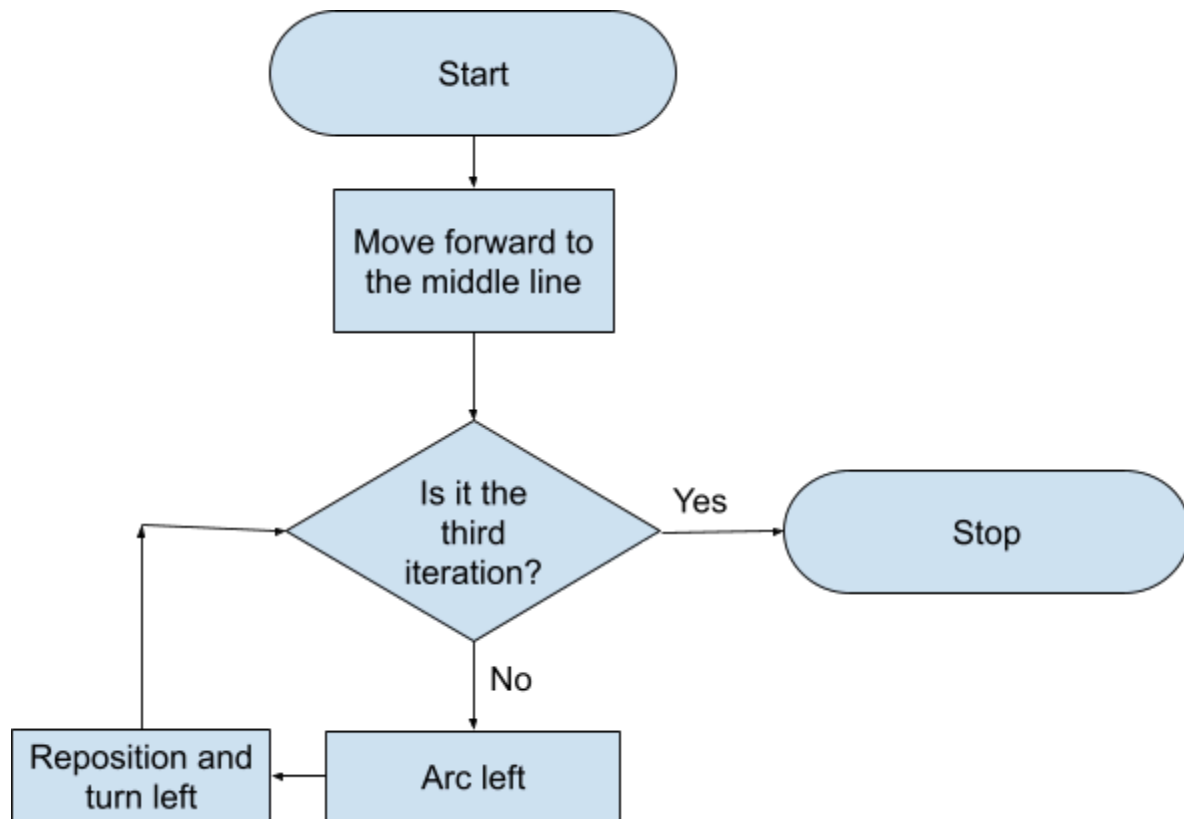


Figure C3. Sketch that was sent to RPL to laser print.

Appendix D: Flowchart



Appendix E: Code

References: Claude used for debugging and syntax. Color detection code copied from lab. No other outside sources were used.

```
// function definitions and constants
void forward(float d);
void back(float t);
void turnRight(int theta);
void turnLeft(int theta);
void arcLeft(int theta);
void stop();
void initColor();
int getColor();
int getSide();

// lower bounds for color detection
const int yellowMin = 200;
const int blueMin = 105;

// constants for readability and initializing
const int yellow = 2;
const int blue = 1;
const int black = 0;
int time = 0;
int color = 0;
int startingColor = 0;
int timer = 0;
int side = 0;
const int none = 0;
const int right = 1;
const int left = 2;
const int both = 3;
const int up = 0;
const int down = 1;

// interrupt on pin 2 for color detection
ISR(PCINT2_vect) {
    if (PIND & 0b00000100) {
        TCNT1 = 0; // resets timer to 0 on upward pin change
    }
}
```

```

    else {
        timer = TCNT1; // set timer equal to the period on downward change
    }
}

// main loop :D
int main(void) {
    //initialize
    DDRB = 0b00101111; // init output pins
    initColor(); // init color sensing
    Serial.begin(9600); // activate serial printing

    // hard coded motion for block collection
    forward(24);
    moveArm(down);
    arcLeft(250);
    forward(10);
    turnLeft(220);
    forward(3);
    arcLeft(230);
    forward(10);
    turnLeft(200);
    arcLeft(250);
    forward(5);
    turnLeft(90);
    stop();
}

// motion functions (all of these are the same so ill only comment one)
void forward(float d) {
    PORTB = 0b00001001; // set correct combination of motor signals
    time = (int)(d * 5000.0 / 36.0); // convert from distance input to time
    _delay_ms(time); // forward motion for set amount of time
}

void back(float d) {
    PORTB = 0b00000110;
    time = (int)(d * 5000.0 / 34.0);
    _delay_ms(time);
}

```

```

void turnRight(int theta){
    PORTB = 0b00001010;
    time = (int)(theta * 2000.0 / 320.0);
    _delay_ms(time);
}

void turnLeft(int theta){
    PORTB = 0b00000101;
    time = (int)(theta * 2000.0 / 320.0);
    _delay_ms(time);
}

void arcLeft(int theta) {
    // for loop alternating between forward and turning motion to get an arc
    for (int i = 0; i < theta/10; i++) {
        forward(.6);
        turnLeft(11);
    }
}

void stop() {
    PORTB = 0b00000000; // stop
}

// color functions
void initColor() {
    PCICR = 0b00000100; //interrupts on pin 2
    sei(); // enable interrupts
    TCCR1A = 0b00000000; // normal timer mode
    TCCR1B = 0b00000001; // no prescaler
    timer = 0; // init timer
}

// color detection function. not used in competition
int getColor() {
    PCMSK2 = 0b00000100; // go to interrupt function on pin change
    _delay_ms(10); // wait a bit to make sure the period gets measured
    PCMSK2 = 0b00000000; // disable the interrupt
}

```

```

int period = 160000 / (timer * 2); // convert to seconds from cycles
// 0 = black, 1 = blue, 2 = yellow
// compares period against our known bounds and returns the color
if (period > yellowMin) {
    return yellow;
}
else if (period > blueMin) {
    return blue;
}
else {
    return black;
}
}

// edge detection. also not used in competition
int getSide() {
    // had to do this annoying delay to distinguish between a single edge
    // and the end condition for milestone 4. makes the motion so much
    // worse but this is the only thing that worked consistently.
    int trial1 = PIND;
    _delay_ms(100);
    int trial2 = PIND;
    int pins = trial1 & trial2; // i have no idea why this works. | makes
    // so much more sense but & was the only thing that functioned as
    // intended. idk.

    // determines which sides the black border is on
    if ((pins & 0b00011000) == 0b00011000) {
        // other pins on this register accept inputs so had to do this to
        // isolate correct pins
        return both;
    }
    else if (pins & 0b00001000) {
        return left;
    }
    else if (pins & 0b00010000) {
        return right;
    }
    else {
        return none;
    }
}

```

```

    }
}

// arm functions
void moveArm(int direction) {
    int elapsed = 0;
    time = 500; // time required to move the arm completely up/down
    // setting pulse width for pwm depending on input
    int pulse = 1;
    if (direction == up) {
        pulse = 2;
    }

    while (elapsed < time) {
        // pwm for servo motion. Sets high for the correct pulse width
        // in a 20ms period, as required by the servo.
        PORTB = 0b00100000;
        _delay_ms(pulse);
        PORTB = 0b00000000;
        _delay_ms(20 - pulse);
        elapsed += 20;
    }
}

```